

Digital Electronics

Part 4 - Binary Representation of

Information

Stéphane Le Goff

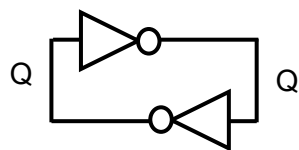
2026

4. Synchronous Sequential Logic Circuits

Combinational logic circuits have the property that the output of a logic block is only a function of the current input values. Still, virtually all useful systems require storage of state information, leading to another class of circuits called *sequential logic circuits*.

In these circuits, the output depends not only on the current values of the inputs, but also on preceding input values. In other words, a sequential logic circuit remembers some of the past history of the system: it has memory.

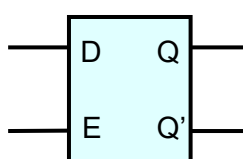
The basic storage element employed in sequential logic circuits is simply designed using a pair of inverters. This is the structure employed to design static random-access memories (SRAMs), D latches, as well as D flip-flops.



Hereafter, we will focus on the design of *synchronous sequential circuits* which constitute by far the most widely used category of sequential circuits. In these circuits, all changes in the output signals must occur at the same time, which correspond to the positive (or negative) edge of a reference signal called *clock*.

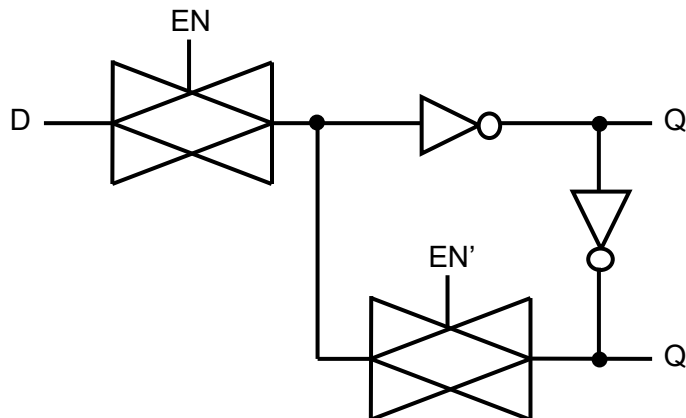
• The D latch

The D latch is a storage element with two inputs D (data) and EN (enable) and two outputs Q and its inverse Q'. Its symbol and truth table are given below.



EN	Q_n	Q_n'
0	Q_{n-1}	Q_{n-1}'
1	D	D'

The most common technique to build a D latch involves the use of two inverters and two transmission gates, as shown below.



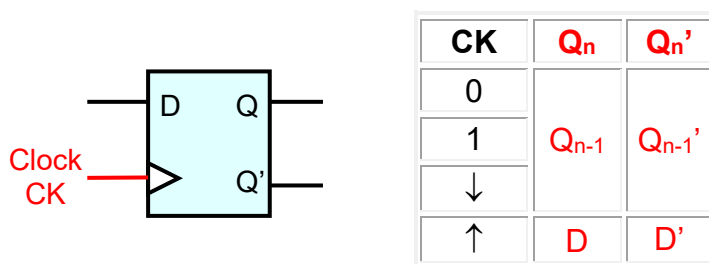
Note that the D latch is an asynchronous storage device since the output Q can change at any time when $EN = 1$.

The D latch is a level-sensitive circuit that passes the D input to the Q output when the EN input is 1. This latch is then said to be in the *transparent* mode.

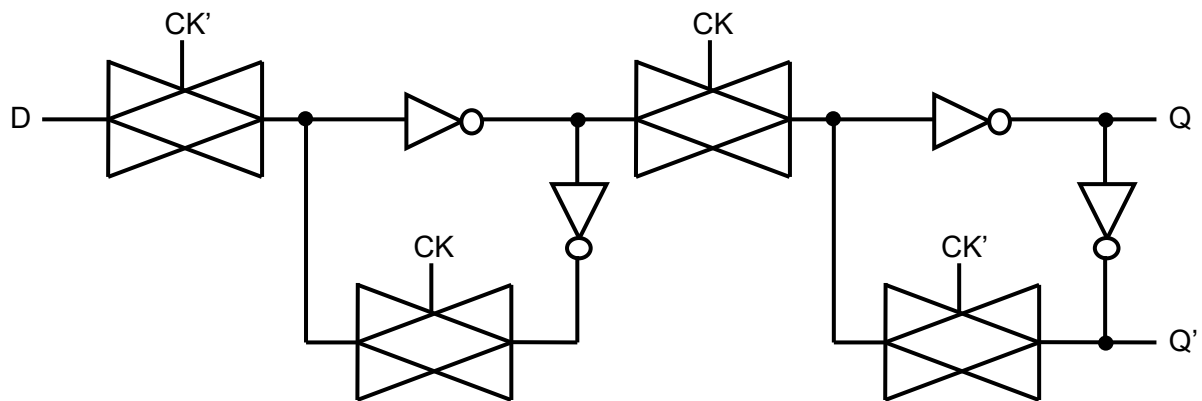
When the EN input is 0, the input data sampled on the falling edge of the EN input is held stable at the output as long as EN remains equal to 0. The latch is then said to be in *hold* mode.

• The D Flip-Flop

The basic storage element required to design a synchronous sequential circuit is the D flip-flop. Its symbol and truth table are given below.

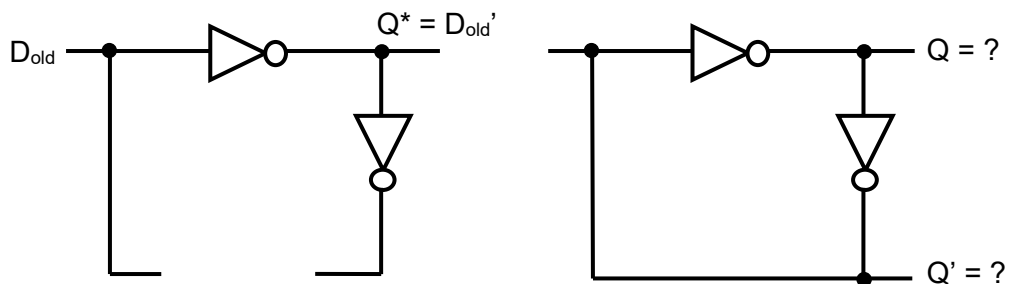


The D flip-flop is designed by cascading two D latches.



Contrary to latches, D flip-flops, also known as *edge-triggered registers*, only sample the input on a clock transition: $0 \rightarrow 1$ for a positive edge-triggered register and $1 \rightarrow 0$ for a negative edge-triggered register. Hereafter, for simplicity sake, we will always assume the use of a positive edge-triggered register. The way D flip-flops work is described below.

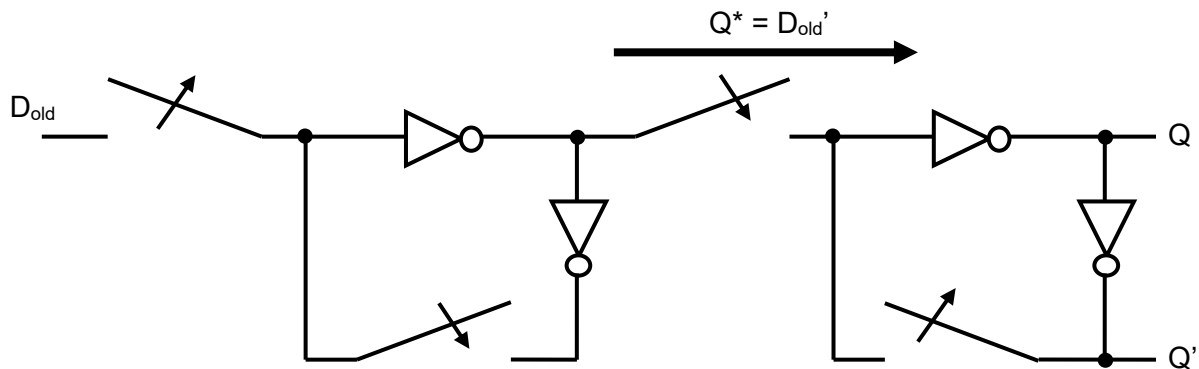
- When $CK = 0$:



D_{old} is the data bit present at the D flip-flop input. Its inverse D_{old}' goes through the first latch ($Q^* = D_{old}'$) since the latter is in the transparent mode, but does not propagate through the second latch which is in hold mode.

The outputs Q and Q' are unknown as we have no idea about the data previously stored by the second latch. Any change in the data bit D_{old} does not affect the values of Q and Q' since there is no direct path between D_{old} and Q .

- During the rising edge of CK:

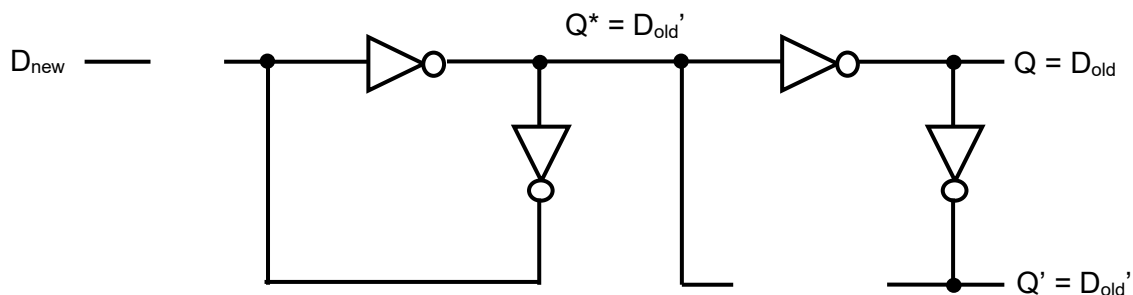


During a rising edge of the clock CK, the first latch switches from the transparent mode to the hold mode, whereas the second latch does the opposite. As a result, the data bit $Q^* = D_{old}'$, present at the second latch input, is passed to the Q output after being inverted.

The Q and Q' outputs are thus updated during this rising edge of CK. This is called the sampling phase.

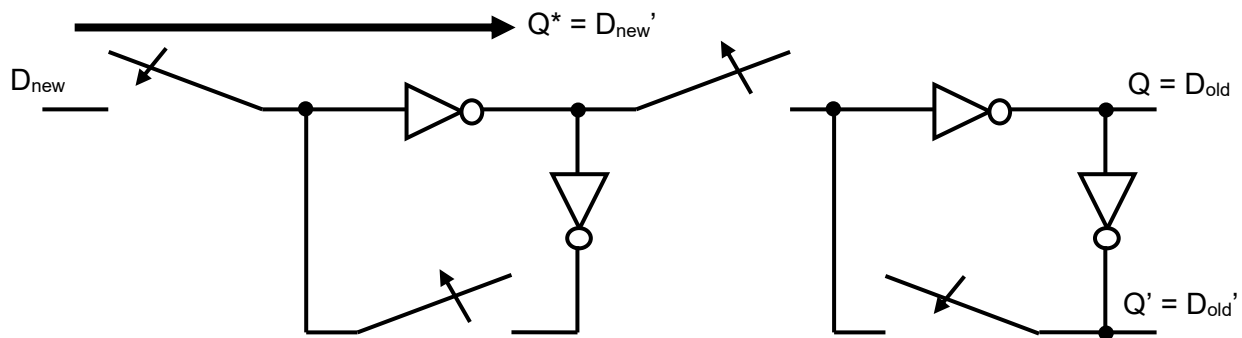
The new values of the D flip-flop outputs are then $Q = (Q^*)' = D_{old}$ and $Q' = D_{old}'$. Since the first latch switches into hold mode, there is still no direct path between the input and output of the D flip-flop.

- When CK = 1:



The data D_{old} sampled on the previous rising edge of CK is now stored by the first latch. Any change in the new data bit D_{new} , present at the D flip-flop input, does not affect the Q and Q' outputs.

- During the falling edge of CK:



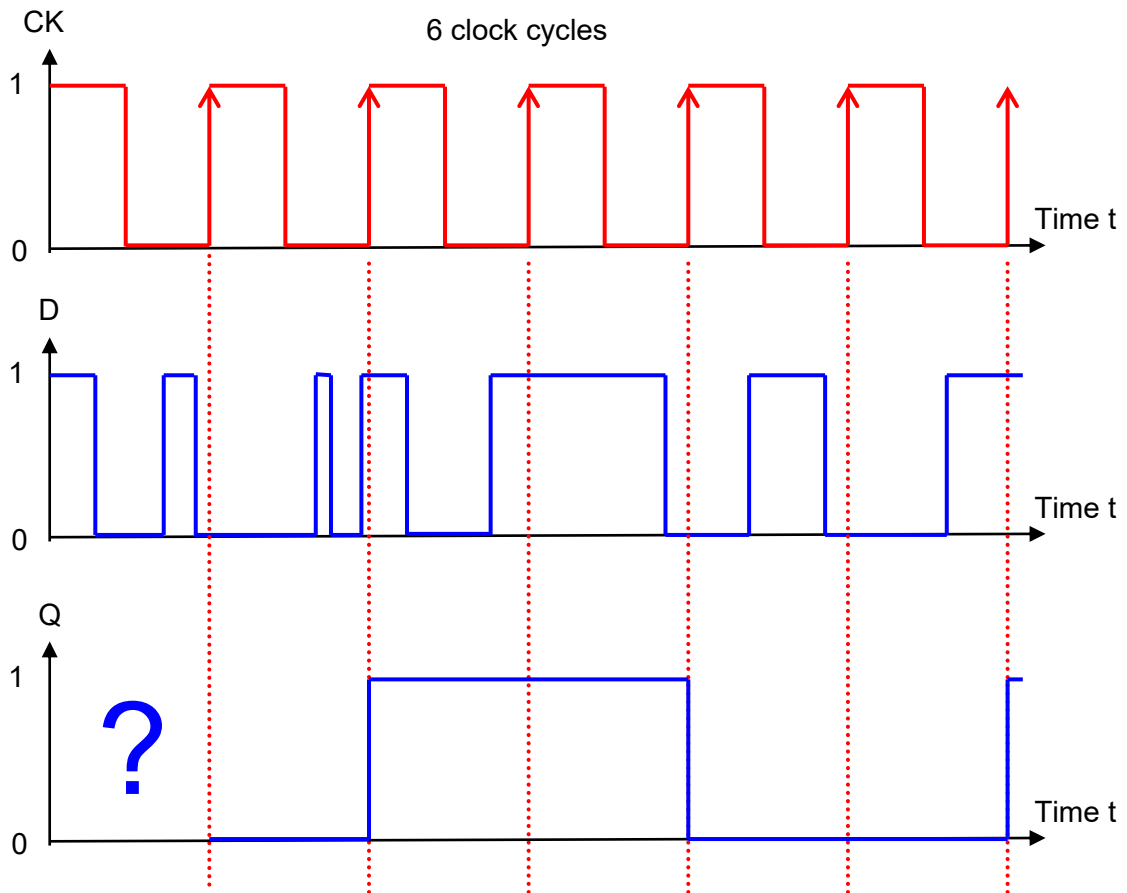
During a falling edge of CK, the second latch switches from the transparent mode to the hold mode, whereas the first latch does the opposite. The inverse of the data bit D_{new} goes through the first latch during this process ($Q^* = D_{\text{new}}'$). However, by the time D_{new}' reaches the input of the second latch, the latter has already entered the hold mode. Hence, D_{new}' does not have time to propagate any further, and the Q and Q' outputs keep their old values ($Q = D_{\text{old}}$ and $Q' = D_{\text{old}}'$).

There is no sufficient time for a data bit to propagate through two latches in a single clock edge. Therefore, any change in the Q and Q' outputs can occur only if the data bit was already present at the input of the second latch at the time of the clock edge. This is the reason why the sampling process only happens during the rising edge of CK, and not during the falling edge of CK.

The purpose of a storage element such as the D flip-flop is to synchronise an eventual change at the output Q with the rising edge of the clock, thus allowing for a “clean” (organised, smooth...) operation of the entire circuit.

D flip-flops are the components that allow synchronous operation in most digital circuits. They are used to control the data flow through the circuit.

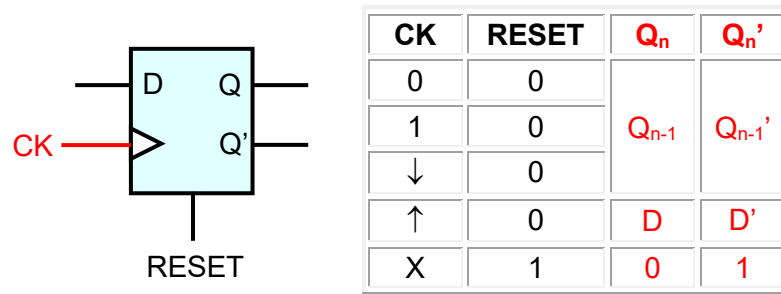
An example of time diagram illustrating the operation of a D flip-flop is given below.



In practice, D flip-flops have (at least) an additional input that can be activated in order to reset the output Q in an asynchronous fashion (i.e., at any time, without having to wait for a rising edge of the clock).

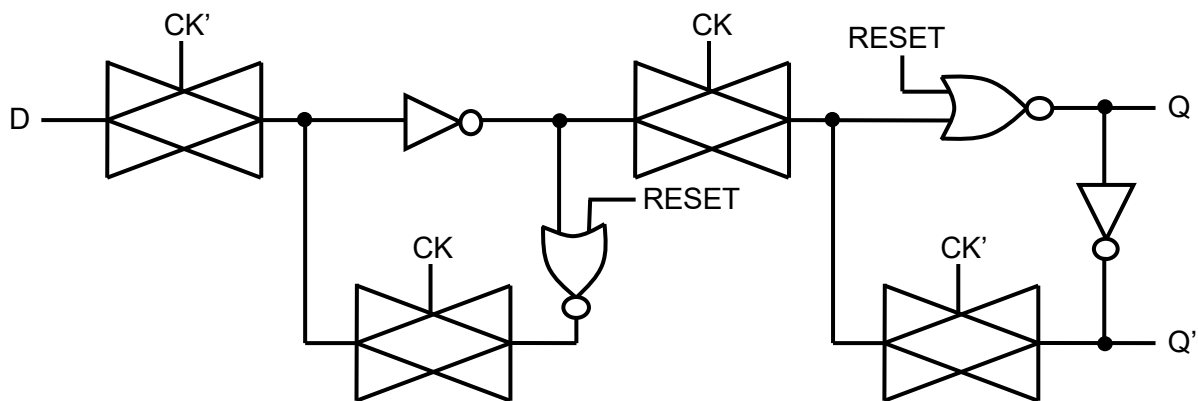
In fact, if a D flip-flop does not have a RESET input, we cannot predict the initial values of the outputs Q and Q' immediately after the power supply has been switched on. This can be a big issue in some applications. In general, we want to ensure that the D flip-flop output is initially equal to zero.

In the truth table shown below, the RESET input is active when equal to one and clearly has priority over the clock (meaning RESET is an asynchronous input).



Note that some flip-flops employ synchronous RESET inputs that can only reset the flip-flop output Q in the presence of a rising edge of the clock.

A D flip-flop with an asynchronous RESET input can be designed as shown below.

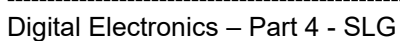


When RESET = 0, both NOR gates act as logic inverters, and the circuit operates like a basic D flip-flop.

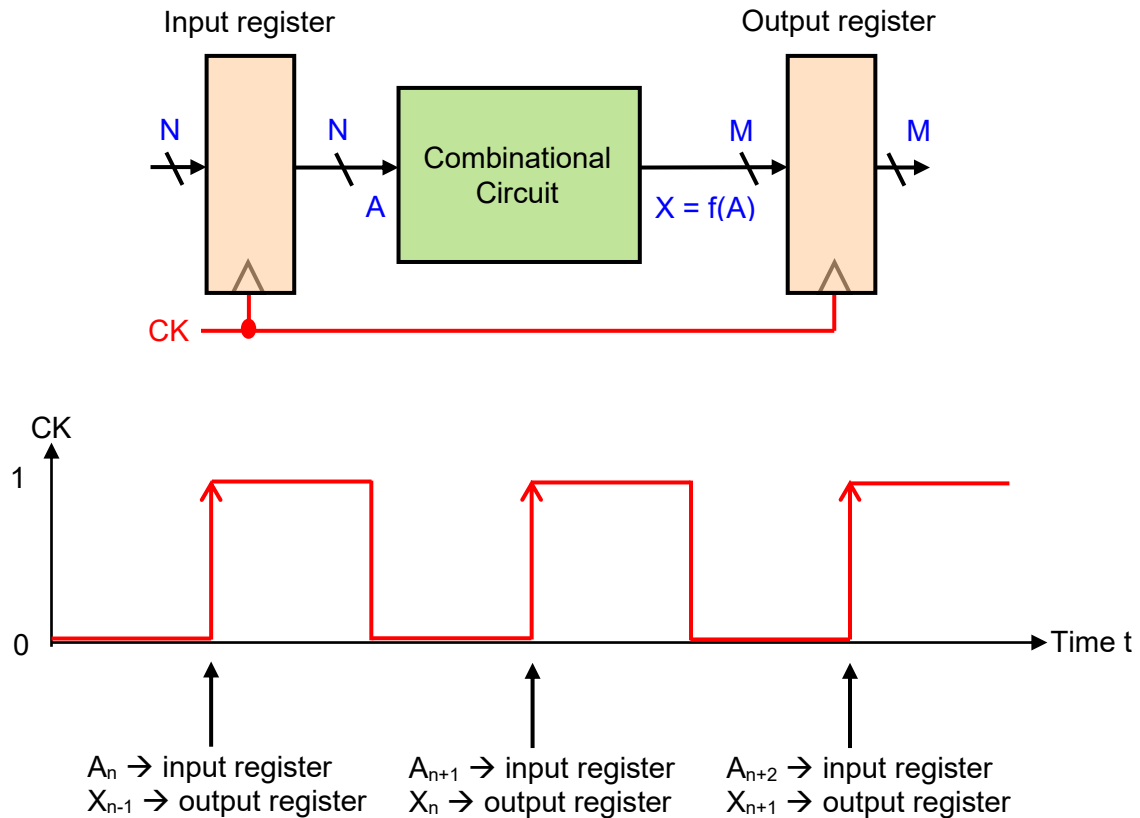
When RESET = 1, the outputs of both NOR gates are always equal to zero, and the circuit is then as shown below.



A 4-bit register suitable for use in the design of synchronous circuits, and simply designed using four D flip-flops, is shown below. Clearly, additional flip-flops could be employed to create a register of any required length. Note that a D flip-flop is a 1-bit register.



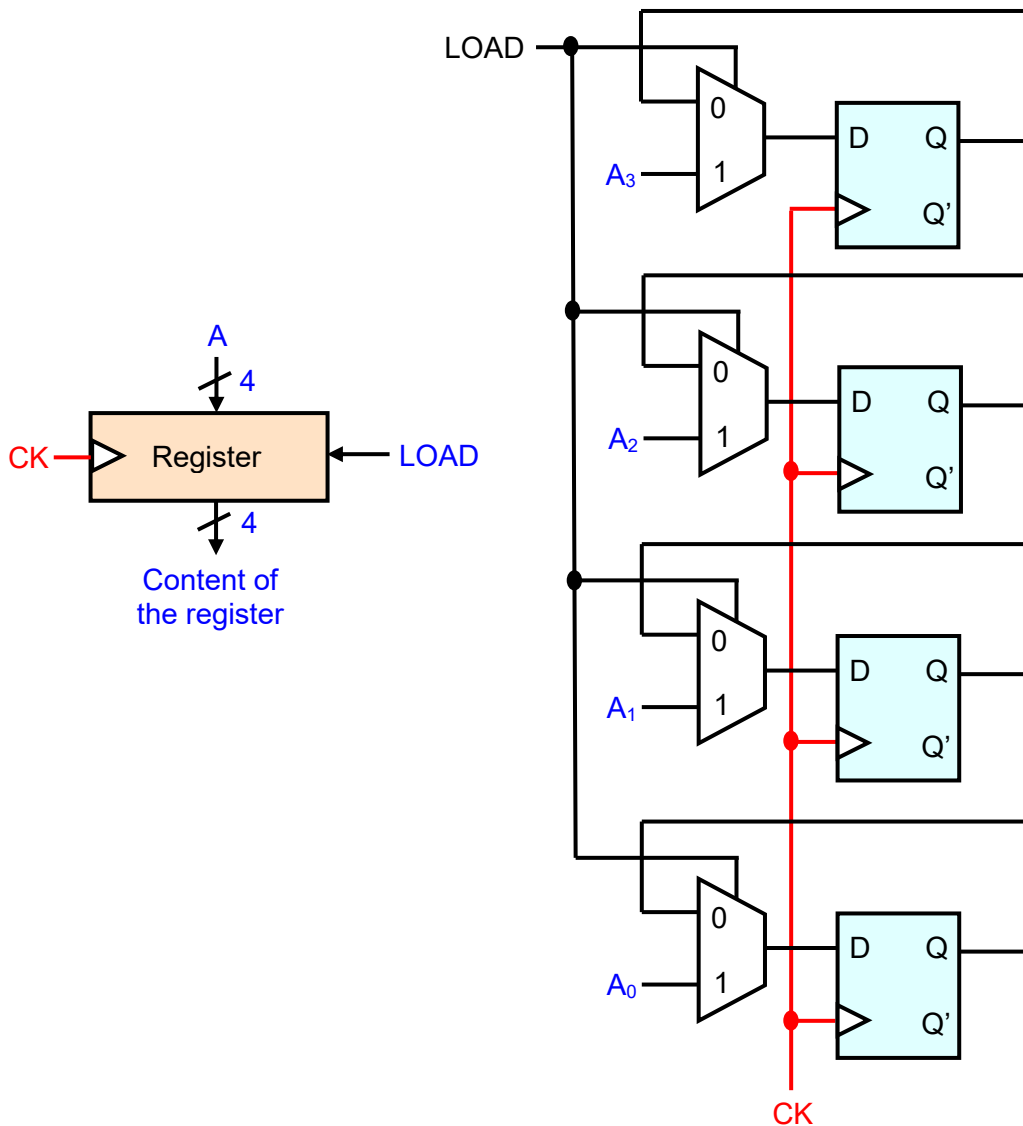
Registers must in fact be used in any practical logic circuit as they are the elements that control the flow of data through the circuit. To illustrate this point, one can consider the generic circuit shown below that employs two registers called input and output registers.



In synchronous sequential circuits, switching events take place concurrently in response to a clock stimulus. Results of operations initiated at time $n \cdot T$ (where n is an integer and T is the clock period) await the next clock transition at time $(n+1) \cdot T$ before progressing to the next processing stage. In this way, data can flow through the circuit in an organised manner.

• Registers with Load Input

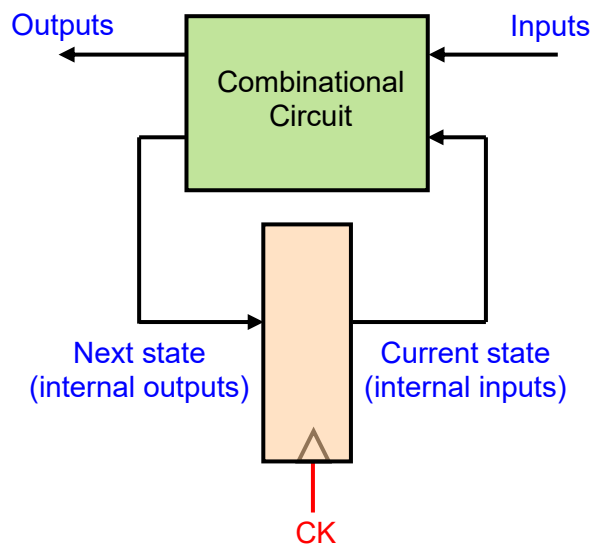
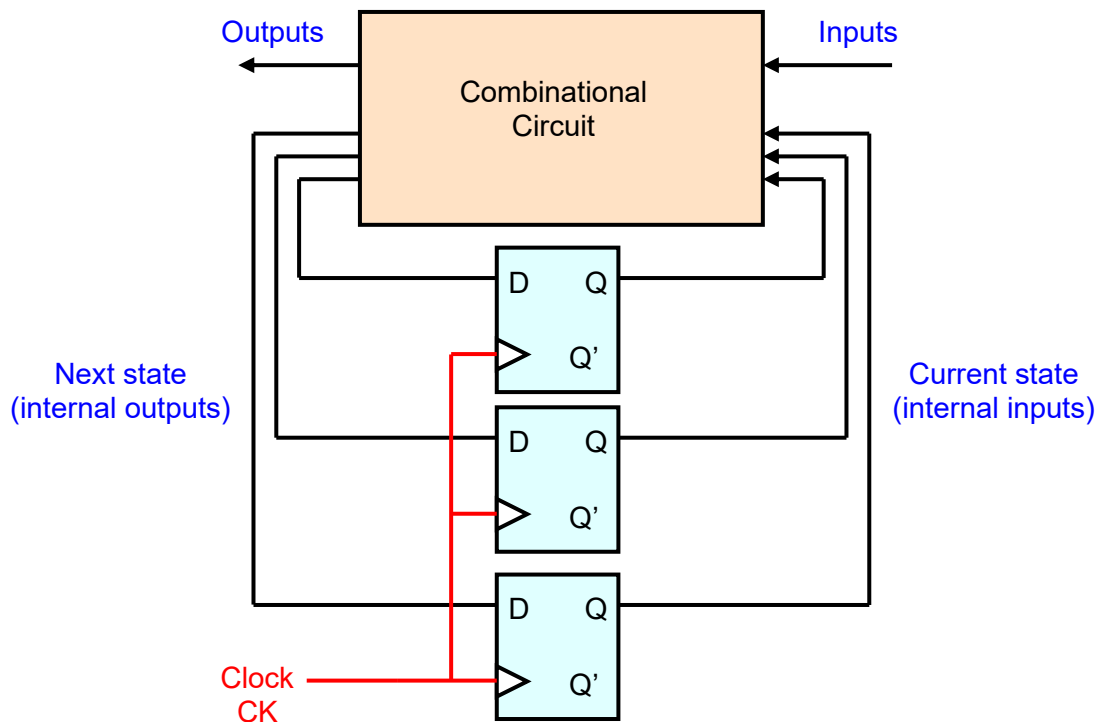
It is often necessary to employ registers that can hold data, even in the presence of a positive clock edge. They can be designed as shown below, in the case of a 4-bit register.



As long as the LOAD input remains equal to 0, the content of the register cannot be updated in response to any clock stimulus. A new data can be stored inside the register only when $\text{LOAD} = 1$. Of course, the sampling of this new data can then only occur on a positive clock edge.

• Design of Synchronous Sequential Circuits

The generic structure of a synchronous sequential circuit is given below. It is composed of a combinational circuit and a register, i.e. a set of D flip-flops.



The register holds the current state of the circuit. The outputs of the circuit are a function of the current external inputs and the current state which in fact represents internal inputs (not seen by the user, but necessary to the proper operation of the circuit). These internal inputs were generated by the circuit during the previous clock cycle.

Note that the particular circuit depicted above can have up to $2^3 = 8$ different states since there are three flip-flops (3-bit register).

The next state is determined based on the current state and the current inputs and is fed to the input of the register. On the rising edge of the clock, the next state bits are copied to the register output and thus become the current state bits.

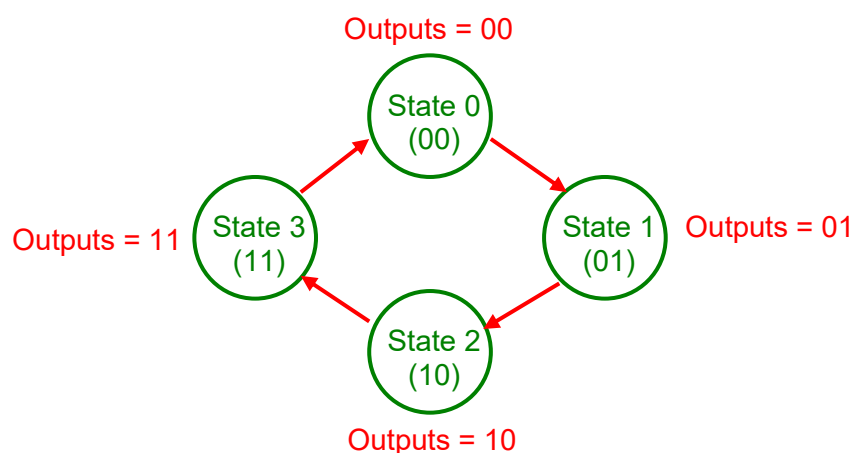
All flip-flops are controlled by the same clock. This is why this type of logic sequential circuit is known as synchronous circuit.

We now consider several examples of synchronous sequential circuits in order to illustrate the design techniques for such circuits.

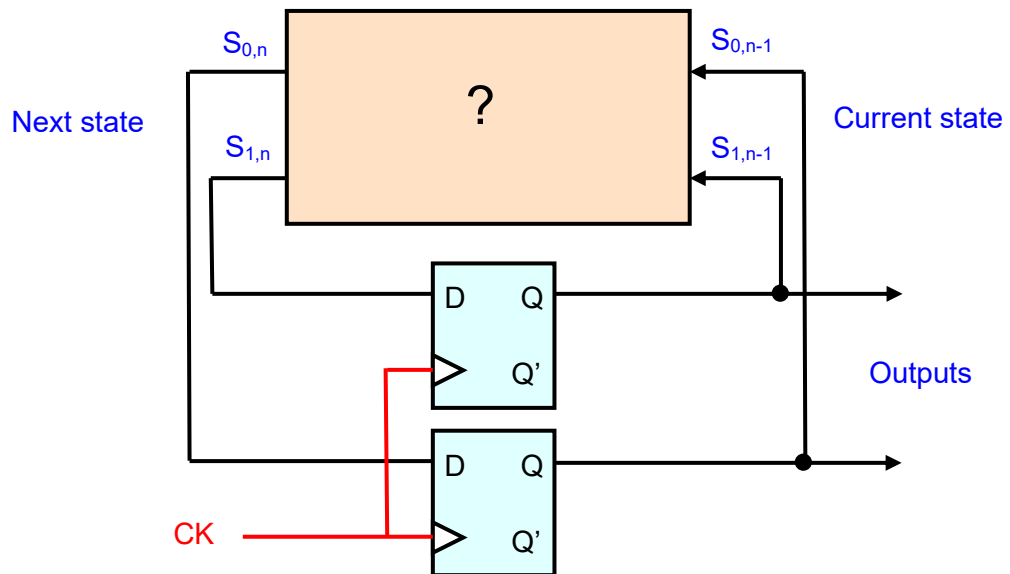
• Design of a 2-Bit Counter

Let us consider the design of a circuit, with no input (except the clock signal), that generates the sequence $00 \rightarrow 01 \rightarrow 10 \rightarrow 11$ and back to 00 again. This can be seen as a sequence $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0 \dots$ in decimal.

Such circuit is a two-bit counter whose operation can be fully described using the following state diagram.



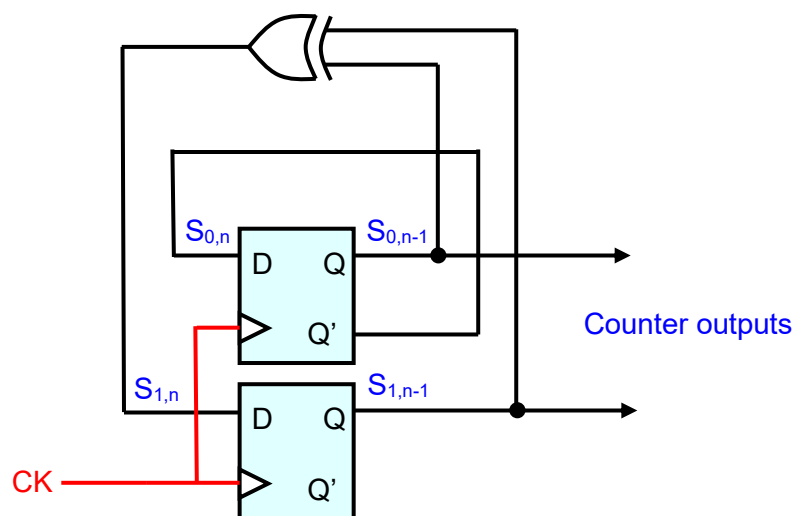
We observe that, in the case of a counter, the outputs are equal to the state of the counter circuit, which simplifies the design process. Since there are only four states, we need to use two flip-flops to store the state of the circuit, i.e. the internal inputs.



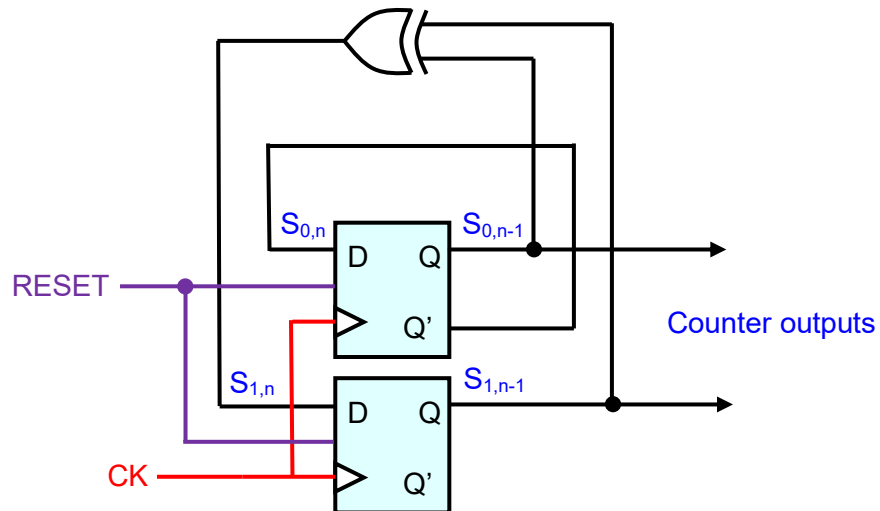
The truth table and the corresponding logic circuit are shown below.

$S_{1,n-1}$	$S_{0,n-1}$	$S_{1,n}$	$S_{0,n}$
0	0	0	1
0	1	1	0
1	0	1	1
1	1	0	0

$$S_{1,n} = S_{1,n-1} \oplus S_{0,n-1} \text{ and } S_{0,n} = \overline{S_{0,n-1}}$$



Note that, in practice, it is preferable to employ D- flip-flops with an asynchronous RESET input to ensure that the counting process can be started from zero at any desired time. In this case, the circuit is as shown below.

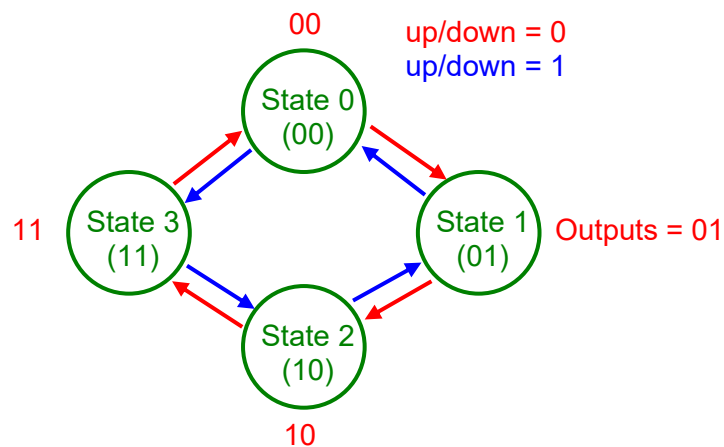


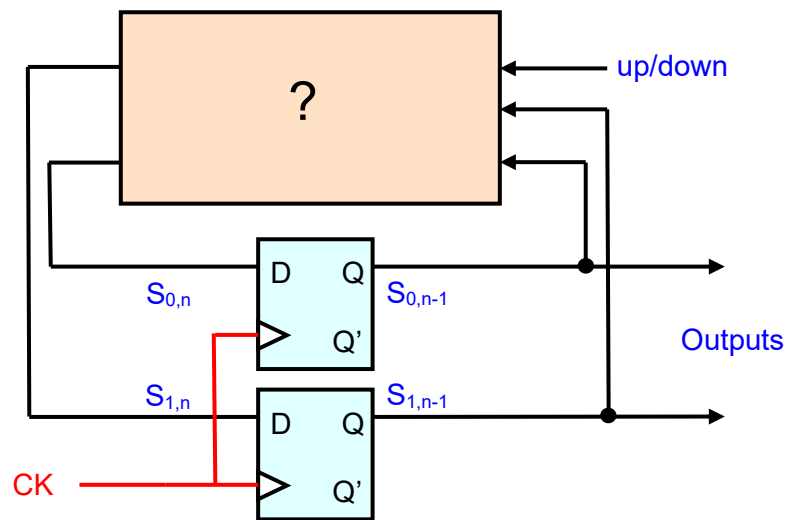
• Design of an Up-Down 2-Bit Counter

Let us now complicate the previous circuit by adding an input, called up/down, that operates as follows:

- If up/down = 0, the circuit generates the sequence 00 → 01 → 10 → 11 and back to 00;
- If up/down = 1, the circuit generates the sequence 00 → 11 → 10 → 01 and back to 00.

The operation of this circuit is described using the state diagram given below.





The truth table of this circuit is shown below.

$S_{1,n-1}$	$S_{0,n-1}$	u/d (up/down)	$S_{1,n}$	$S_{0,n}$
0	0	0	0	1
0	0	1	1	1
0	1	0	1	0
0	1	1	0	0
1	0	0	1	1
1	0	1	0	1
1	1	0	0	0
1	1	1	1	0

The Karnaugh maps are as follows:

		$S_{1,n-1} \ S_{0,n-1}$			
		00	01	11	10
u/d	0	1			1
	1	1			1

$S_{0,n} = \overline{S_{0,n-1}}$

$S_{0,n-1}'$

		$S_{1,n-1} S_{0,n-1}$			
		00	01	11	10
u/d	0		1		1
	1	1		1	

$$S_{1,n} = \overline{S_{1,n-1}}\overline{S_{0,n-1}}u/d + \overline{S_{1,n-1}}S_{0,n-1}\overline{u/d} + S_{1,n-1}\overline{S_{0,n-1}}u/d + S_{1,n-1}S_{0,n-1}\overline{u/d}$$

$$S_{1,n} = (\overline{S_{1,n-1}}\overline{S_{0,n-1}} + S_{1,n-1}S_{0,n-1}) u/d + (\overline{S_{1,n-1}}S_{0,n-1} + S_{1,n-1}\overline{S_{0,n-1}}) \overline{u/d}$$

$$S_{1,n} = (\overline{S_{1,n-1}} \oplus S_{0,n-1}) u/d + (S_{1,n-1} \oplus S_{0,n-1}) \overline{u/d}$$

$$S_{1,n} = (S_{1,n-1} \oplus S_{0,n-1}) \oplus u/d$$

